Make a plot with both redshift and universe age axes using astropy.cosmology

Authors

Neil Crighton, Stephanie T. Douglas

Learning Goals

- Plot relationships using `matplotlib`
- Add a second axis to a `matplotlib` plot
- Relate distance, redshift, and age for two different types of cosmology using `astropy.cosmology`

Keywords

units, physics, cosmology, matplotlib

Summary

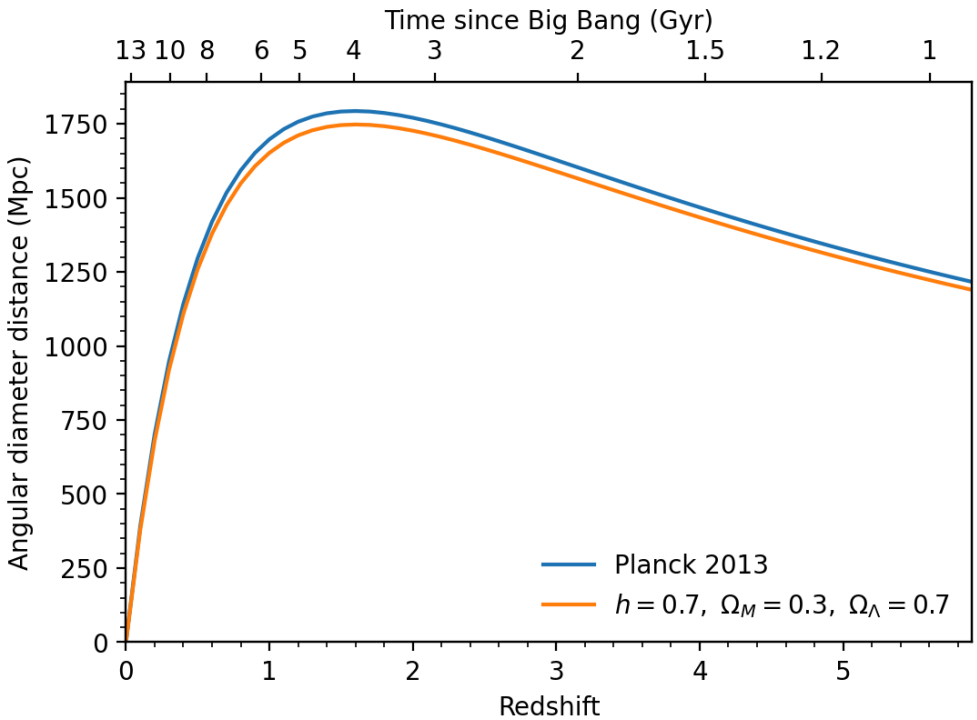
Each redshift corresponds to an age of the universe, so if you're plotting some quantity against redshift, it's often useful show the universe age too. The relationship between the two changes depending the type of cosmology you assume, which is where `astropy.cosmology` comes in. In this tutorial we'll show how to use the tools in `astropy.cosmology` to make a plot like this:

```
# Set up matplotlib
import matplotlib.pyplot as plt

%matplotlib inline
```

```
from IPython.display import Image

Image(filename="ang_dist.png", width=500)
```



We start with a cosmology object. We will make a flat cosmology (which means that the curvature density $\Omega_k = 0$) with a hubble parameter of 70 km/s/Mpc and matter density $\Omega_M = 0.3$ at redshift 0. The `FlatLambdaCDM` cosmology then automatically infers that the dark energy density Ω_Λ must $= 0.7$, because $\Omega_M + \Omega_\Lambda + \Omega_k = 1$.

```
from astropy.cosmology import FlatLambdaCDM
import astropy.units as u

# In this case we just need to define the matter density
# and hubble parameter at z=0.

# Note the default units for the hubble parameter H0 are km/s/Mpc.
# We will pass in a `Quantity` object with the units specified.

cosmo = FlatLambdaCDM(H0=70 * u.km / u.s / u.Mpc, Om0=0.3)
```

Note that we could instead use one of the built-in cosmologies, like `WMAP9` or `Planck13`, in which case we would just redefine the `cosmo` variable.

Now we need an example quantity to plot versus redshift. Let's use the angular diameter distance, which is the physical transverse distance (the size of a galaxy, say) corresponding to a fixed angular separation on the sky. To calculate the angular diameter distance for a range of redshifts:

```
import numpy as np

zvals = np.arange(0, 6, 0.1)
dist = cosmo.angular_diameter_distance(zvals)
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[4], line 4
      1 import numpy as np
      3 zvals = np.arange(0, 6, 0.1)
----> 4 dist = cosmo.angular_diameter_distance(zvals)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
   1249 """Angular diameter distance in Mpc at a given redshift.
   1250
   1251 This gives the proper (sometimes called 'physical') transverse
   (...)   1272 .. [3] Peebles, P. (1993). Principles of Physical Cosmology, pp 325-
   1273 """
   1274 z = aszarr(z)
-> 1275 return self.comoving_transverse_distance(z) / (z + 1.0)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
   1183 @deprecated_keywords("z", since="7.0")
   1184 def comoving_transverse_distance(self, z):
   1185     r"""Comoving transverse distance in Mpc at a given redshift.
   1186
   1187     This value is the transverse comoving distance at redshift ``z``
   (...)   1207     This quantity is also called the 'proper motion distance' in som
   1208     """
-> 1209     return self._comoving_transverse_distance_z1z2(0, z)

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
   1212 r"""Comoving transverse distance in Mpc between two redshifts.
   1213
   1214 This value is the transverse comoving distance at redshift ``z2`` as
   (...)   1234 This quantity is also called the 'proper motion distance' in some te
   1235 """
   1236 Ok0 = self.Ok0
-> 1237 dc = self._comoving_distance_z1z2(z1, z2)
   1238 if Ok0 == 0:
   1239     return dc

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    392 # Use np.sqrt here to handle negative s (Om0>1).
    393 prefactor = self.hubble_distance / np.sqrt(s * self.Om0)
    394 return prefactor * (
--> 395     self._T_hypergeometric(s / (z1 + 1.0))
    396     - self._T_hypergeometric(s / (z2 + 1.0))
    397 )

File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
    399 def _T_hypergeometric(self, x, /):
    400     r"""Compute value using Gauss Hypergeometric function 2F1.
    401
    402     .. math::
   (...)    417        in a flat cosmology. MNRAS, 468(1), 927-930.
    418     """
--> 419     return 2 * np.sqrt(x) * hyp2f1(1.0 / 6, 1.0 / 2, 7.0 / 6, -(x**3))
```

```
File /opt/hostedtoolcache/Python/3.12.11/x64/lib/python3.12/site-packages/astropy/co
     26 def hyp2f1(*args, **kwargs):
---> 27     raise ModuleNotFoundError("No module named 'scipy.special'")

ModuleNotFoundError: No module named 'scipy.special'
```

Note that we passed an array of redshifts to `cosmo.angular_diameter_distance` and it produced a corresponding array of distance values, one for each redshift. Let's plot them:

```
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist)
```

To check the units of the angular diameter distance, take a look at the unit attribute:

```
dist.unit
```

Now let's put some age labels on the top axis. We're going to pick a series of round age values where we want to place axis ticks. You may need to tweak these depending on your redshift range to get nice, evenly spaced ticks.

```
ages = np.array([13, 10, 8, 6, 5, 4, 3, 2, 1.5, 1.2, 1]) * u.Gyr
```

To link the redshift and age axes, we have to find the redshift corresponding to each age. The function `z_at_value` does this for us.

```
from astropy.cosmology import z_at_value

ageticks = [z_at_value(cosmo.age, age) for age in ages]
```

Now we make the second axes, and set the tick positions using these values.

```
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist)
ax2 = ax.twiny()
ax2.set_xticks(ageticks)
```

We have ticks on the top axis at the correct ages, but they're labelled with the redshift, not the age. We can fix this by setting the tick labels by hand.

```
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist)
ax2 = ax.twiny()
ax2.set_xticks(ageticks)
ax2.set_xticklabels(["{:g}".format(age) for age in ages.value])
```

We need to make sure the top and bottom axes have the same redshift limits. They may not line up properly in the above plot, for example, depending on your setup (the age of the universe should be ~13 Gyr at z=0).

```
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist)
ax2 = ax.twiny()
ax2.set_xticks(ageticks)
ax2.set_xticklabels(["{:g}".format(age) for age in ages.value])
zmin, zmax = 0.0, 5.9
ax.set_xlim(zmin, zmax)
ax2.set_xlim(zmin, zmax)
```

We're almost done. We just need to label all the axes, and add some minor ticks. Let's also tweak the y axis limits to avoid putting labels right near the top of the plot.

```
fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist)
ax2 = ax.twiny()
ax2.set_xticks(ageticks)
ax2.set_xticklabels(["{:g}".format(age) for age in ages.value])
zmin, zmax = 0, 5.9
ax.set_xlim(zmin, zmax)
ax2.set_xlim(zmin, zmax)
ax2.set_xlabel("Time since Big Bang (Gyr)")
ax.set_xlabel("Redshift")
ax.set_ylabel("Angular diameter distance (Mpc)")
ax.set_ylim(0, 1890)
ax.minorticks_on()
```

Now for comparison, let's add the angular diameter distance for a different cosmology, from the Planck 2013 results. And then finally, we save the figure to a png file.

```
from astropy.cosmology import Planck13

dist2 = Planck13.angular_diameter_distance(zvals)

fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot(zvals, dist2, label="Planck 2013")
ax.plot(zvals, dist, label="$h=0.7,\ \Omega_M=0.3,\ \Omega_\Lambda=0.7$")
ax.legend(frameon=0, loc="lower right")
ax2 = ax.twiny()
ax2.set_xticks(ageticks)
ax2.set_xticklabels(["{:g}".format(age) for age in ages.value])
zmin, zmax = 0.0, 5.9
ax.set_xlim(zmin, zmax)
ax2.set_xlim(zmin, zmax)
ax2.set_xlabel("Time since Big Bang (Gyr)")
ax.set_xlabel("Redshift")
ax.set_ylabel("Angular diameter distance (Mpc)")
ax.minorticks_on()
ax.set_ylim(0, 1890)
fig.savefig("ang_dist.png", dpi=200, bbox_inches="tight")
```

`bbox_inches='tight'` automatically trims any whitespace from around the plot edges.

And we're done!

Exercise

Well, almost done. Notice that we calculated the times on the upper axis using the original cosmology, not the new cosmology based on the Planck 2013 results. So strictly speaking, this axis applies only to the original cosmology, although the difference between the two is small. As an exercise, you can try plot two different upper axes, slightly offset from each other, to show the times corresponding to each cosmology. Take a look at the first answer to this question on Stack Overflow for some hints on how to go about this.